

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №8

по дисциплине «Естественно-языковой интерфейс интеллектуальных
систем»

на тему «Разработка автоматизированной системы синтеза речи
(вариант 1)»

Выполнили студенты гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Крапивин Ю. Б.

Минск 2026

Цель: освоить на практике основные принципы создания систем синтеза речи.

Задание (вариант 1): разработать автоматизированную систему синтеза речи со следующей комбинацией признаков варианта:

- **Поддерживаемые языки** – английский;
- **Предметная область** – научные статьи по computer science.

Система синтеза должна обеспечивать следующие минимальные возможности:

- ввод текста, или копирование через буфер обмена, или поддержку указателя мыши в любом другом приложении;
- воспроизведение сгенерированного речевого сигнала для введённого текста;
- настройки (выбор) голоса диктора, темпа, громкости и других параметров для чтения.

В отчёт включены: описание структуры разработанной системы, технологий и методик; основные алгоритмы и принципы реализации компонент системы (блок-схемы алгоритмов); результаты тестирования; результаты анализа полученных данных и предложения по улучшению работы системы.

Синтез речи реализован не как отдельное приложение, а как ещё один сценарий использования уже существующей информационно-поисковой системы (ЛР1–ЛР4): озвучить можно любой текст, с которым система работает – документ коллекции из предметной области варианта (научная статья по computer science), реферат, перевод, а также произвольный текст, введённый вручную или скопированный из любого другого приложения через буфер обмена. Для синтеза выбрана нейросетевая библиотека с открытым исходным кодом Piper, работающая локально, без обращения к облачным сервисам (см. раздел «Технологии и методики»).

Структура разработанной системы

Синтез речи реализован как новый сценарий использования поверх архитектуры ЛР1–ЛР4 (см. docs/ARCHITECTURE.md): границы сервисов по-прежнему проведены по зоне ответственности, а не по лабораторной работе. Синтез речи тянет за собой тяжёлую зависимость (среда исполнения нейросетей `onnxruntime`) и отдельно скачиваемые файлы голосов, поэтому, по тому же принципу изоляции тяжёлых ML-зависимостей, что и `lang-id-service` (ЛР2) и `mt-service` (ЛР4), он выделен в новый микросервис `speech-service`. Базе данных для синтеза речи ничего хранить не

(`SpeechSynthesisService`) не зависит от HTTP-деталей: клиент `speech-service` (`SpeechServiceClient`) передаётся ему как зависимость, поэтому в тестах подменяется заглушкой;

– `frontend` (React, TypeScript) – хранит настройки голоса, режет текст на фрагменты, запрашивает у `api` звук каждого фрагмента и проигрывает его через Web Audio API. Состоит из трёх контекстов и трёх компонентов, перечисленных в таблице 1.

Таблица 1 – Компоненты интерфейса, реализующие синтез речи

Компонент	Назначение
<code>SpeechSettingsContext</code>	Настройки голоса, темпа, громкости; сохраняются в <code>localStorage</code> браузера и переживают перезагрузку страницы
<code>SpeechPlaybackContext</code>	Очередь фрагментов: запрос звука, декодирование, воспроизведение, предвыборка следующего фрагмента, остановка
<code>speechChunking.ts</code>	Разбиение текста на фрагменты для озвучивания
<code>SpeakButton</code>	Кнопка «Озвучить / Остановить» рядом с любым текстом: документом, рефератом, переводом; подсвечивается звуковой волной, пока звучит её текст
<code>ClipboardSpeakButton</code>	Кнопка «Озвучить из буфера»: читает текст из буфера обмена и озвучивает его
Вкладка «Синтез речи» страницы «Настройки»	Выбор голоса, ползунки темпа и громкости, поле произвольного текста для проверки

Технологии и методики

Выбор технологии синтеза. Методичка предлагает FreeTTS, Microsoft Speech API, Google Speech API, Web Speech API и нейросетевые библиотеки. Все варианты, кроме последнего, имеют существенные ограничения для этой системы: Microsoft Speech API привязан к Windows, тогда как система развёртывается одной командой в Docker и не должна зависеть от операционной системы хоста; облачные API требуют ключей, доступа в интернет и передачи текста документов третьей стороне; Web Speech API отдаёт то голоса, что установлены в конкретной операционной системе и браузере, поэтому результат невозможно воспроизвести на другой машине, и набор голосов не контролируется системой; FreeTTS даёт заметно более механическое звучание. Поэтому выбрана нейросетевая система **Piper** (`piper-tts 1.8.0`): голоса представляют собой модели VITS (вариационный автокодировщик с состязательным обучением), экспортированные в формат ONNX, качества *medium* (частота дискретизации 22 050 Гц, моно, 16 бит).

Синтез идёт на CPU быстрее реального времени (см. раздел «Результаты тестирования»), работает без интернета, а набор голосов задан образом контейнера и потому одинаков на любой машине.

Голоса. Для варианта 1 (английский язык) доступно 10 голосов одного качества (*medium*): 7 американских (Amy, Lessac, HFC Female, Kristin – женские; Ryan, HFC Male, Joe – мужские) и 3 британских (Alba и Jenny – женские, Alan – мужской). Голос по умолчанию – `en_US-amy-medium`.

Темп. В Piper скорость речи задаётся параметром `length_scale` – множителем длительности фонем. Пользователь выбирает темп r от $0,5\times$ до $2,0\times$ с шагом $0,1$; в `length_scale` он переводится обратным значением:

$$\text{length_scale} = \frac{1}{r}, \quad (1)$$

поэтому темп $2\times$ сокращает звучание вдвое, а $0,5\times$ – удваивает его, не меняя высоты голоса (в отличие от простого ускорения воспроизведения готовой записи). Точность этого соотношения проверена измерением (табл. 4).

Громкость. Громкость регулируется на стороне браузера узлом `GainNode` Web Audio API (коэффициент от 0 до 1): она не требует повторного синтеза, поэтому изменение громкости не влияет на нагрузку на сервер.

Потоковый WAV. Piper выдаёт звук по предложениям, а общий размер звука до окончания синтеза неизвестен. Поэтому `speech-service` отправляет WAV-заголовок с размером-заглушкой `0xFFFFFFFF` (принятая практика для потокового WAV), а затем блоки PCM по мере готовности. Браузеры не всегда корректно декодируют такой заголовок, поэтому интерфейс, получив ответ, заменяет поля размера в заголовке на реальные значения.

Воспроизведение через Web Audio API, а не через элемент audio. Chrome разрешает воспроизведение звука только в ответ на действие пользователя (нажатие); при озвучивании по фрагментам следующий фрагмент запускается уже не в обработчике нажатия, а после завершения предыдущего, и элемент `audio` в этот момент мог бы быть заблокирован. Контекст `AudioContext` достаточно один раз «разбудить» по нажатию, после чего все последующие фрагменты играют без ограничений.

Основные алгоритмы

Общая схема обмена. Последовательность обмена сообщениями при синтезе одного фрагмента текста показана на рисунке 2. Ключевое свойство схемы – потоковость на всём пути: звук первого предложения возвращается интерфейсу, пока сервис ещё синтезирует следующие.

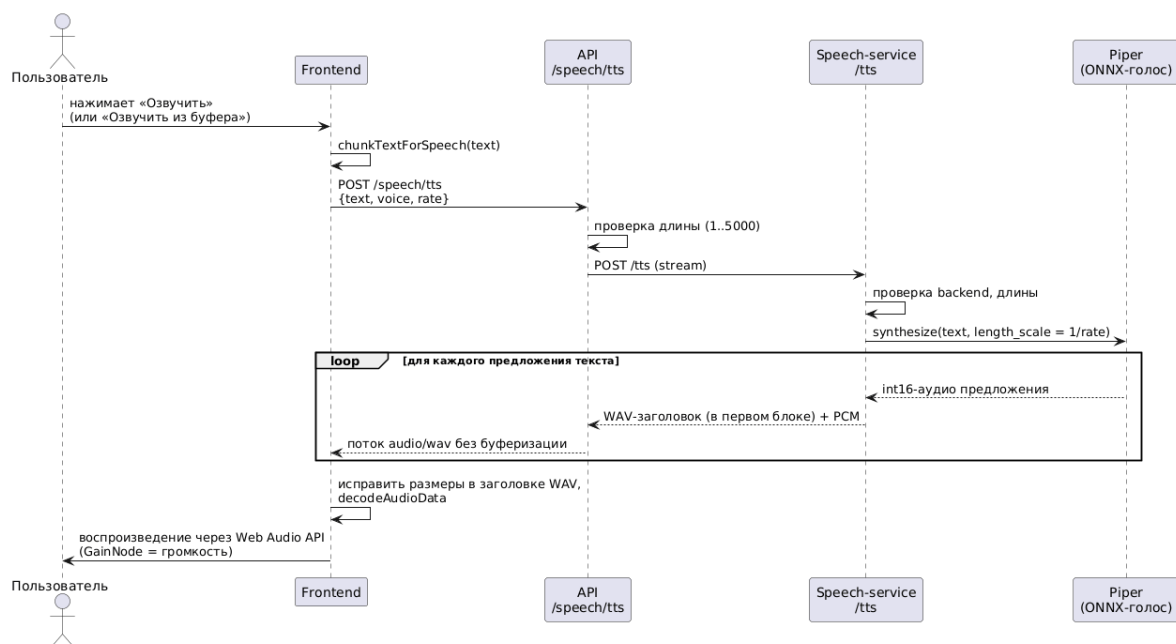


Рисунок 2 – Обмен сообщениями при синтезе речи

Разбиение текста на фрагменты. Научная статья – это несколько тысяч символов, а синтез всего текста одним запросом заставил бы пользователя ждать, пока будет готов звук всей статьи (измерено в разделе «Результаты тестирования»). Поэтому текст режется на фрагменты, и озвучивание начинается сразу после синтеза первого. Алгоритм (рисунок 3) собирает фрагменты из целых предложений: предложения по очереди добавляются в текущий фрагмент, пока его длина не превысит целевую (120 символов); предложение длиннее максимума (240 символов) режется жёстко. Границы фрагментов совпадают с границами предложений, поэтому интонация не ломается посреди фразы, а максимальная длина фрагмента остаётся намного ниже серверного лимита в 5000 символов.

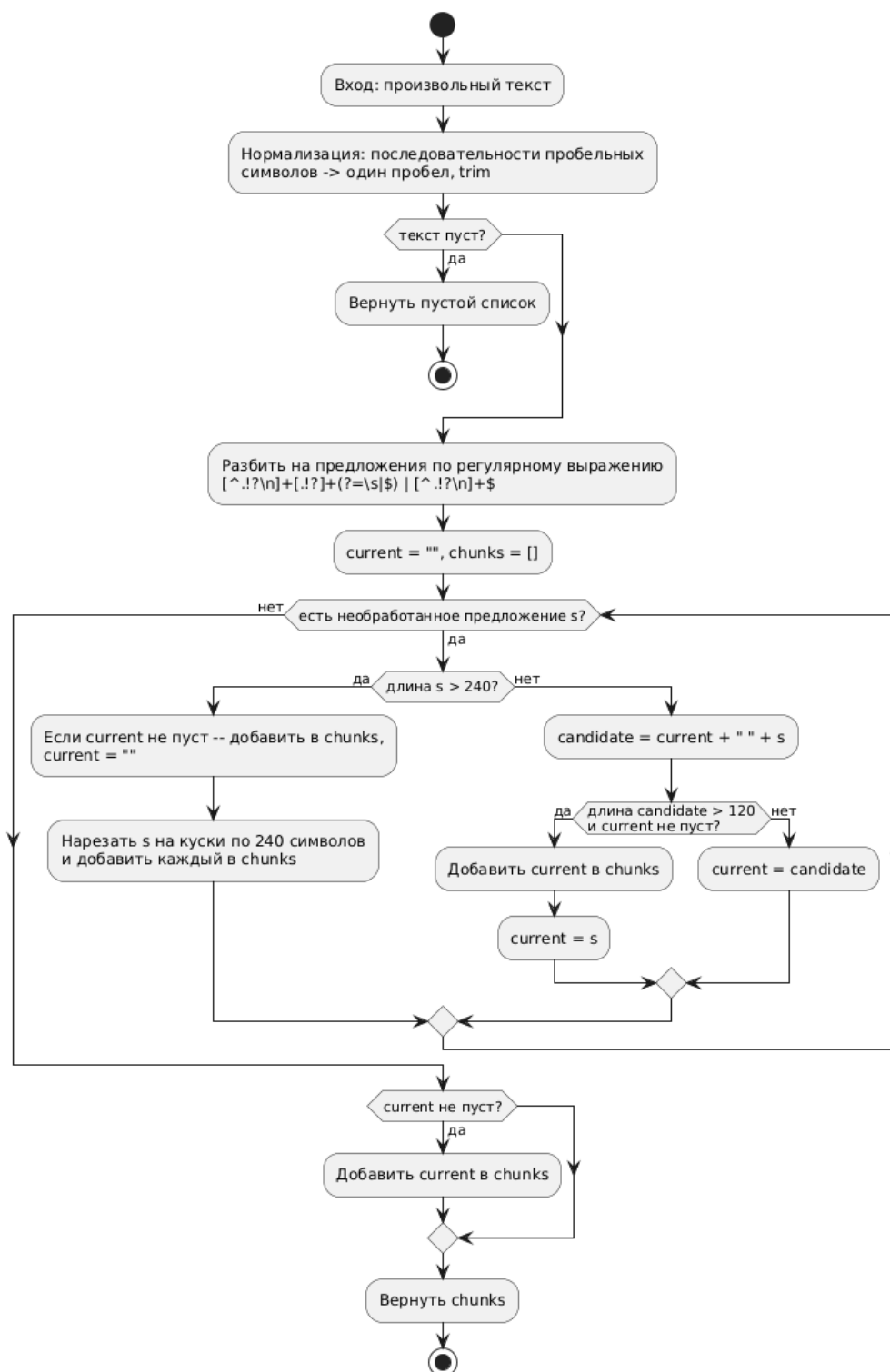


Рисунок 3 – Разбиение текста на фрагменты для озвучивания

Воспроизведение с упреждающей выборкой. Пока звучит фрагмент i , запрос на синтез фрагмента $i + 1$ уже отправлен (рисунок 4). Если синтез быстрее звучания, а это так, пока синтез идёт быстрее реального времени, то пауз между фрагментами нет вообще: к моменту окончания фрагмента i звук фрагмента $i + 1$ уже готов. Остановка пользователем пре-

рывает все незавершённые запросы через `AbortController`, чтобы сервер не тратил ресурсы на синтез уже ненужного текста.



Рисунок 4 – Воспроизведение фрагментов с упреждающей выборкой

Способы ввода текста и пользовательский интерфейс

Методичка допускает три способа получить текст для озвучивания: ввод, копирование через буфер обмена либо указатель мыши в другом приложении. Реализованы первые два:

– **Ввод.** Произвольный текст вводится в поле на вкладке «Синтез речи» страницы «Настройки». Кроме того, кнопка «Озвучить» расположена рядом с каждым текстом, с которым работает система: документом коллекции (диалог просмотра/редактирования документа – текст научной статьи из предметной области варианта), рефератом (ЛР3, вкладки алгоритмического, TextRank- и эмбединг-методов) и результатом машинного перевода (ЛР4);

– **Буфер обмена.** Кнопка «Озвучить из буфера» читает текст через `navigator.clipboard.readText()` и озвучивает его, одновременно подставляя в поле проверки, чтобы пользователь видел, что именно читается. Так можно озвучить текст из любого другого приложения или страницы: выделить, скопировать (Ctrl+C) и нажать кнопку. Если браузер не разрешает доступ к буферу, а также если буфер пуст, интерфейс сообщает об этом;

– **Указатель мыши в другом приложении** не реализован: веб-страница не может получить выделенный текст из чужого окна – это ограничение безопасности браузера, а не системы. Его роль выполняет буфер обмена, разрешённый заданием как равноправная альтернатива.

Параметры чтения меняются на той же вкладке: голос (список из 10 голосов), темп (ползунок 0,5–2,0×) и громкость (ползунок 0–100%). Настройки применяются к следующему озвучиванию сразу, без перезагрузки; до нажатия на кнопку «Озвучить» их можно проверить на примере текста в том же поле. Помимо кнопок, озвучиванием можно управлять голосом (команды «read this» и «stop reading», см. ЛР9).

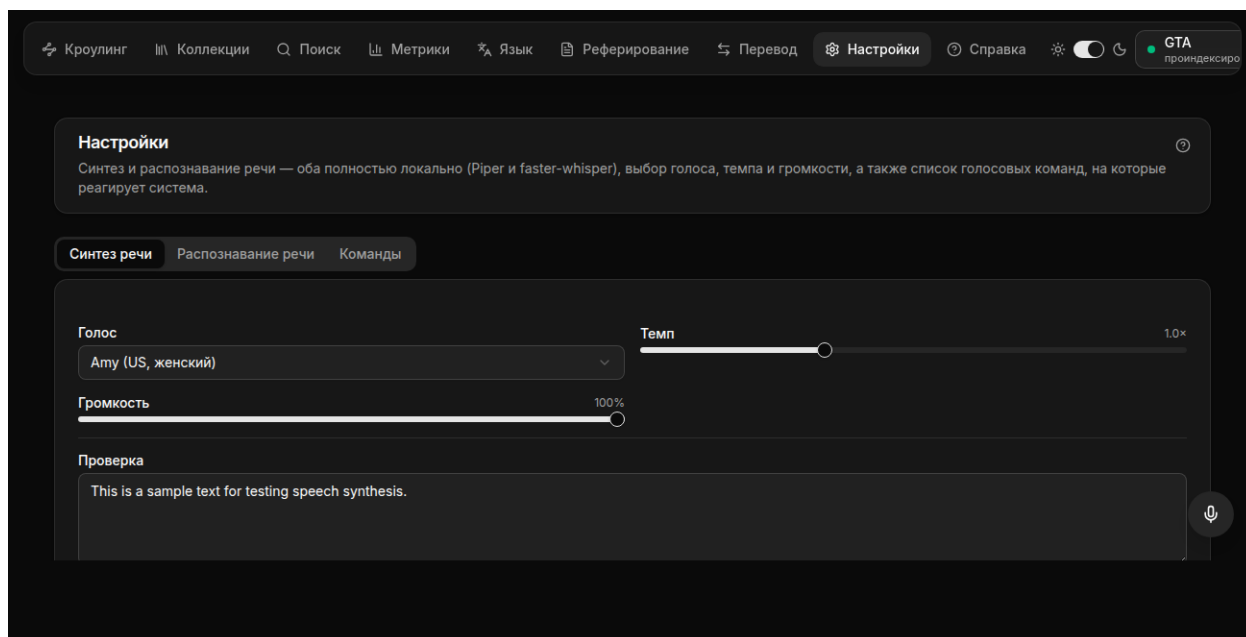


Рисунок 5 – Вкладка «Синтез речи» страницы «Настройки»: голос, темп, громкость, поле текста, кнопки «Озвучить из буфера» и «Озвучить»

Результаты тестирования

Тестирование проводилось на работающей системе (`docker compose`), через тот же эндпоинт `POST /speech/tts`, которым пользуется интерфейс; контейнер `speech-service` был ограничен четырьмя ядрами CPU и 4 ГиБ памяти (см. «Структура разработанной системы»). Тексты – предложения из области варианта (научные статьи по computer science). Каждое измерение повторялось трижды после «прогрева» голоса, в таблицах приведены средние значения. Под *коэффициентом реального времени* (RTF) понимается отношение времени синтеза к длительности получившейся речи: $RTF < 1$ означает, что синтез быстрее звучания.

Голоса. Сравнение всех десяти голосов на одном тексте (234 символа, два предложения) приведено в таблице 2. Голоса заметно различаются темпом речи по умолчанию (12,1–15,1 с на один и тот же текст), но скорость синтеза у них практически одинакова: RTF от 0,46 до 0,53 (в среднем 0,48), то есть синтез примерно вдвое быстрее звучания. Первое обращение к голосу дороже последующих на 3,0–4,4 с (в среднем 3,8 с): это время загрузки модели в память.

Таблица 2 – Синтез одного текста (234 символа) разными голосами

Голос	Регион, пол	Речь, с	1-й звук, с	Синтез, с	RTF	1-й вызов, с
Amy	US, ж	14,9	3,3	7,0	0,47	10,9
Lessac	US, ж	14,0	3,1	6,6	0,47	10,2
HFC Female	US, ж	12,6	3,2	6,5	0,52	9,9
Kristin	US, ж	14,6	3,3	6,7	0,46	11,1
Ryan	US, м	12,1	2,9	6,4	0,53	9,4
HFC Male	US, м	14,1	3,4	6,8	0,48	11,2
Joe	US, м	12,7	3,4	6,3	0,50	9,8
Alba	UK, ж	12,5	3,5	6,2	0,50	10,4
Jenny	UK, ж	14,2	3,5	6,5	0,46	10,3
Alan	UK, м	15,1	3,5	6,9	0,46	11,0

Длина текста. Зависимость времени синтеза от длины текста (голос Amy) приведена в таблице 3. Время до *первого звука* не зависит от длины текста (2,7 с для всех текстов, кроме одного предложения из двух, где оно 3,3 с): сервис отдаёт звук по мере синтеза каждого предложения, а не после всего текста. Полное время синтеза растёт линейно с длиной (RTF около 0,48).

Таблица 3 – Синтез текстов разной длины (голос Аму)

Символов	1-й звук, с	Синтез, с	Речь, с	RTF
64	2,7	2,7	4,2	0,63
234	3,3	7,1	14,7	0,48
700	2,7	21,4	44,5	0,48
1532	2,7	45,6	96,2	0,47

Целесообразность разбиения на фрагменты. Для текста в 2298 символов (три раза повторённый реферат статьи, около 2 мин 23 с речи) интерфейс при синтезе одним запросом получил бы первый звук через 63,6 с: браузер декодирует WAV только целиком, поэтому воспроизведение не может начаться, пока ответ не получен полностью (хотя сервер и начинает отдавать данные уже через 2,7 с). После разбиения на 19 фрагментов длиной 78–152 символа первый звук слышен через 2,9 с – в 22 раза быстрее. Время синтеза каждого следующего фрагмента (2,5–4,1 с) во всех случаях меньше длительности звучания предыдущего (5,2–9,8 с), минимальный запас – 1,6 с, поэтому при упреждающей выборке пауз между фрагментами не возникло ни разу (из 18 стыков).

Темп. Результат изменения темпа на том же тексте приведён в таблице 4. Темп монотонно и предсказуемо меняет длительность речи, однако не строго обратно пропорционально: при $r = 2,0$ звучание сокращается в 1,62 раза (а не в 2), при $r = 0,5$ увеличивается в 1,75 раза (а не в 2). Вероятная причина – паузы между предложениями и вокруг знаков препинания, которые в Piper не масштабируются параметром `length_scale`; подтверждение этого предположения требует отдельного исследования и здесь не проводилось. Синтез при повышенном темпе не дороже (время синтеза падает вместе с длительностью речи).

Таблица 4 – Влияние темпа на длительность речи (голос Аму, текст 234 символа)

Темп r	Речь, с	Длительность к $r = 1$	Ожидалось ($1/r$)	Фактическое ускорение
0,5	25,9	1,75	2,00	0,57×
0,75	18,4	1,24	1,33	0,81×
1,0	14,8	1,00	1,00	1,00×
1,25	13,0	0,88	0,80	1,14×
1,5	10,7	0,72	0,67	1,38×
2,0	9,2	0,62	0,50	1,62×

Потребление памяти. Измерялось по счётчику памяти контейнера `speech-service` (`anon` в `cgroup`) после перезапуска и прогрева моделей распознавания (ЛР9): 435 МиБ без единого голоса Piper. Каждый вновь загруженный голос добавляет 130–190 МиБ (в среднем около 150 МиБ); после однократного обращения ко всем десяти голосам контейнер занимал

1957 МиБ. В ходе длинных синтезов (тексты в тысячи символов) занятая память продолжила расти, и по данным `docker stats` к концу серии измерений достигла 3,84 ГиБ из 4 ГиБ лимита (см. «Анализ результатов»).

Разборчивость синтезированной речи. Качество звука оценивается прежде всего слушателем, а автоматическая проверка может дать лишь косвенную оценку. В качестве такой оценки использован круговой тест: 10 предложений из области варианта (научные статьи по computer science; в них – термины *cosine similarity*, *hyperplane*, *lemmatization* и др.) озвучены тремя голосами (Amy, Ryan, Alba) и распознаны моделью Whisper (`small`, подробнее – в отчёте по ЛР9). Доля ошибочных слов (WER) составила 2,6%, при этом 24 из 30 озвученных предложений распознаны полностью без ошибок. Голоса Amy и Ryan дали по 2,0%, британский голос Alba – 3,9%. Все ошибки пришлись на редкие термины (*lemmatization* → *limitization*, *hyperplane* → *hyperblane*) и на артикль в начале фразы («The term frequency...» вместо «Term frequency...»), то есть текст в целом разборчив, а слабое место – произношение специальной лексики. Ограничение методики: модель распознавания сама допускает ошибки и в норме ошибается на редких словах независимо от того, как они произнесены, поэтому результат – оценка снизу, а не заменитель прослушивания.

Автоматические тесты. Слой `speech-service` покрыт тестами маршрутов (`services/speech-service/tests/test_routes.py`): проверка состояния, потоковая передача байтов синтезатора, отказ при неизвестном бэкенде, при пустом тексте и при тексте длиннее лимита; слой `application` в `api` – тестами `test_speech_application.py` (передача потока от клиента, пробрасывание ошибки вышестоящего сервиса). Внешний синтезатор в этих тестах заменяется заглушкой, поэтому они проверяют контракты, а не качество звука. Полный набор тестов речевого слоя (синтез и распознавание) прошёл без ошибок: 9 тестов `speech-service` и 18 тестов `api` (`test_speech_application.py`, `test_speech_domain.py`, `test_speech_streaming_domain.py`).

Анализ результатов и предложения по улучшению

Что показали измерения.

а) *Скорости синтеза достаточно для озвучивания по фрагментам.* Синтез идёт примерно вдвое быстрее звучания (RTF около 0,48) на четырёх ядрах CPU, без видеокарты. Поэтому упреждающая выборка полностью скрывает время синтеза: на тексте из 19 фрагментов между фрагментами не возникло ни одной паузы, а запас составил не менее 1,6 с.

б) *Разбиение на фрагменты необходимо.* Без него первое слово в тексте из 2298 символов прозвучало бы через минуту с лишним, а с ним –

через 2,9 с.

в) *Основная задержка – до первого звука.* Даже для одного короткого предложения пользователь ждёт 2,7–3,5 с: столько синтезируется первое предложение (Piper выдаёт звук только после синтеза предложения целиком), плюс к этому для первого обращения к голосу – ещё около 3,8 с на загрузку модели.

г) *Голоса практически равноценны по скорости и разборчивости,* различаясь в основном естественным темпом речи; на темп влияет не только параметр `length_scale`.

д) *Память – главный ограничитель.* Каждый загруженный голос добавляет около 150 МиБ и никогда не выгружается; после долгих синтезов на разных голосах контейнер приблизился к лимиту в 4 ГиБ (3,84 ГиБ), а при исчерпании лимита ядро может завершить процесс, что остановит и синтез, и распознавание речи (ЛР9), находящиеся в одном контейнере.

Ограничения системы.

– Нет озвучивания текста из чужого приложения по указателю мыши (ограничение браузерной среды, см. выше); вместо него – буфер обмена.

– Голоса только английские качества *medium*: они звучат ровно, но заметно синтетически; интонация не зависит от смысла предложения.

– Специальная лексика научных статей (аббревиатуры, формулы, обозначения вроде $O(n \log n)$, названия методов) читается по общим правилам произношения и может быть прочитана неправильно.

– Громкость применяется к фрагменту в момент начала его воспроизведения: изменение ползунка во время звучания фрагмента слышно только со следующего.

– Одновременно допускается одно озвучивание: новое нажатие прерывает предыдущее.

Предложения по улучшению.

а) *Ограничить число одновременно загруженных голосов* (кэш с вытеснением наименее недавно использованного, 2–3 голоса): память контейнера перестанет зависеть от числа посещённых голосов, ценой повторной загрузки модели (около 3,8 с) при возврате к вытесненному голосу. Заодно загружать голос по умолчанию при старте контейнера, как сейчас делается для моделей распознавания, – тогда самое частое первое обращение не будет платить за загрузку.

б) *Уменьшить задержку до первого звука:* делать первый фрагмент короче остальных (одно короткое предложение), а следующие – длиннее; либо проигрывать WAV по мере поступления через **MediaSource**, а не после получения всего фрагмента.

в) *Предобработка научного текста* перед синтезом: раскрытие аббревиатур и обозначений, замена формул словесным описанием, пропуск ссылок на литературу ([12]), таблиц и колонтитулов.

г) *Голоса высокого качества (high)* для основного голоса и отдельный словарь произношения терминов предметной области.

д) *Показ прогресса озвучивания*: подсвечивать читаемое предложение в тексте и позволять перемотку по предложениям.

е) *Расширение до других языков* (русский, немецкий, французский): архитектура сервиса это допускает – голос задаётся идентификатором, добавляются лишь файлы голосов и записи в список интерфейса.

Использованные готовые компоненты

Таблица 5 – Библиотеки, готовые данные и программные средства, использованные при реализации ЛР8 (дополнительно к перечисленным в отчётах по ЛР1–ЛР4)

Компонент	Назначение
Piper (<code>piper-tts 1.8.0</code>)	Нейросетевой синтез речи на CPU: модели VITS в формате ONNX, потоковая выдача звука по предложениям, параметр <code>length_scale</code> для темпа.
Голоса Piper (<code>rhasspy/piper-voices</code>)	Десять предобученных английских голосов качества <i>medium</i> (<code>en_US</code> : amy, lessac, hfc_female, kristin, ryan, hfc_male, joe; <code>en_GB</code> : alba, jenny_dioco, alan); скачиваются при сборке образа командой <code>python -m piper.download_voices</code> .
<code>onnxruntime</code>	Среда исполнения моделей ONNX (зависимость Piper).
FastAPI, uvicorn, httpx	HTTP-сервер <code>speech-service</code> , потоковый ответ <code>StreamingResponse</code> и клиент <code>api</code> для проксирования потока (уже использовались в предыдущих лабораторных).
Web Audio API (<code>AudioContext</code> , <code>GainNode</code>)	Декодирование и воспроизведение WAV в браузере, регулировка громкости.
Async Clipboard API	Чтение текста из буфера обмена для кнопки «Озвучить из буфера».
Radix UI Slider (<code>shadcn/ui</code>)	Ползунки темпа и громкости на странице «Настройки».

Выводы

В ходе выполнения лабораторной работы реализована автоматизированная система синтеза речи по варианту 1 (английский язык, научные статьи по computer science) как новый сценарий использования поверх архитектуры, разработанной в ЛР1–ЛР4. Синтез вынесен в отдельный микросервис `speech-service` – по тому же принципу изоляции тяжёлых ML-зависимостей, что и определение языка и нейросетевой перевод; базу данных синтез не затрагивает, существующие контракты системы не нарушены.

Обязательные возможности реализованы следующим образом. Ввод текста: поле произвольного текста, кнопка «Озвучить» рядом с документом, рефератом и переводом, а также кнопка «Озвучить из буфера», которая читает текст, скопированный в любом другом приложении (озвучивание по

указателю мыши в чужом окне невозможно в браузерной среде по соображениям безопасности; заменой служит буфер обмена, разрешённый заданием). Воспроизведение: нейросетевой синтез локальной моделью Piper, потоковая передача звука и воспроизведение через Web Audio API. Настройки: 10 английских голосов (7 американских и 3 британских), темп от 0,5× до 2,0× и громкость от 0 до 100 %, которые сохраняются между сеансами.

Ключевое решение – разбиение текста на фрагменты по предложениям с упреждающей выборкой следующего фрагмента: без него озвучивание текста в 2298 символов началось бы через 63,6 с, а с ним – через 2,9 с, причём на длинном тексте пауз между фрагментами не возникает вовсе (синтез идёт в среднем вдвое быстрее звучания, RTF около 0,48). Круговая проверка разборчивости (синтез с последующим распознаванием) дала 2,6 % ошибочных слов на предложениях из предметной области, с ошибками только на редких терминах.

Измерения выявили и слабые места, которые описаны в разделе «Анализ результатов и предложения по улучшению»: задержку до первого звука в 2,7–3,5 с, дополнительные 3,8 с на первое обращение к каждому голосу и отсутствие вытеснения загруженных голосов – после обращения ко всем десяти голосам и длинных синтезов память контейнера приблизилась к лимиту (3,84 из 4 ГиБ). Для каждого из них предложено решение, наиболее важное из которых – кэш голосов с вытеснением.